

Universidade Federal do Ceará Instituto Universidade Virtual Bal. em Sistemas e Mídias Digitais
Disciplina de Programação 2

Prof. Wellington Sarmento

Respostas do Exercício 01 do Livro de Algoritmos

Capítulo 1: Introdução a algoritmos (Pesquisa Binária e Big O)

1.1 Suponha que você tenha uma lista com 128 nomes e esteja fazendo uma pesquisa binária. Qual seria o número máximo de etapas? Resposta: 7 etapas.

Esta pergunta poderia ser reformulada para "Quantas vezes preciso dividir 128 por 2 até chegar a 1, que seria o fim da busca?", haja vista que a busca binária divide a lista de nomes à metade a cada etapa. Teríamos, assim, as etapas: 128 -> 64 -> 32 -> 16 -> 8 -> 4 -> 2 -> 1. Que operação matemática poderíamos usar que nos daria este resultado de forma mais generalizada? O logaritmo na base 2 ($\log_2$ ou, simplesmente, $\log$). A operação com logaritmos trabalha como se estivéssemos fazendo a pergunta como "quantas vezes devo dividir um número por um determinado valor B (base) até chegar a 1?" Desta forma, o $\log_2 4$, por exemplo, seria $4 \div 2 = 2 \div 2 = 1$, resultando em *duas vezes* o processo de divisão por dois até alcançar 1, portanto, $\log_2 4 = 2$.

A pesquisa binária corta a lista pela metade a cada etapa. O número máximo de etapas é o logaritmo na base 2 do tamanho da lista ($\log_2 128 = 7$). Em termos práticos: 128 -> 64 -> 32 -> 16 -> 8 -> 4 -> 2 -> 1.

1.2 Suponha que você duplique o tamanho da lista. Qual seria o número máximo de etapas agora? Resposta: 8 etapas. Como você dobrou o tamanho para 256, a primeira etapa da pesquisa cortará a lista pela metade, voltando para os 128 elementos da questão anterior. Portanto, você só precisa de 1 etapa a mais ($\log_2 256 = 8$).

Exemplo em Javascript (Cálculo de etapas da Pesquisa Binária):

```
// 1. Função de Busca Binária modificada para contar etapas
function pesquisaBinaria(lista, item) {
  let baixo = 0;
  let alto = lista.length - 1;
  let etapas = 0; // Novo contador de etapas

  while (baixo <= alto) {
    etapas++; // Incrementa 1 a cada tentativa de busca

    let meio = Math.floor((baixo + alto) / 2);
    let chute = lista[meio];

    if (chute === item) {
      // Retorna um objeto com o índice e o total de etapas
      return { indice: meio, etapas: etapas };
    }
    if (chute > item) {
      alto = meio - 1;
    }
  }
}
```

```
    } else {
      baixo = meio + 1;
    }
  }

  // Se não encontrar o item, retorna null para o índice e as etapas gastas
  return { indice: null, etapas: etapas };
}

// 2. Funções auxiliares mantidas
function gerarInteiroAleatorio(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}

function gerarListaInteirosAleatorios(tamanho, min, max) {
  const lista = [];
  for (let i = 0; i < tamanho; i++) {
    lista.push(gerarInteiroAleatorio(min, max));
  }
  return lista;
}

// --- TESTANDO O CÓDIGO ---

const tamanhoDaLista = 128; // Experimente mudar para 256, 512, etc.

// Gera a lista aleatória
let minhaLista = gerarListaInteirosAleatorios(tamanhoDaLista, 1, 1000);

// ATENÇÃO: A busca binária exige que a lista esteja ordenada!
minhaLista.sort((a, b) => a - b);

// Vamos escolher um item aleatório que sabemos que está dentro da lista
// para testar
let indiceAleatorioParaBuscar = gerarInteiroAleatorio(0, tamanhoDaLista - 1);
let itemProcurado = minhaLista[indiceAleatorioParaBuscar];

console.log(`Tamanho da lista: ${tamanhoDaLista} itens`);
console.log(`Procurando o número: ${itemProcurado}`);

// Executa a busca
let resultado = pesquisaBinaria(minhaLista, itemProcurado);

if (resultado.indice !== null) {
  console.log(`Item encontrado no índice: ${resultado.indice}`);
} else {
  console.log("Item não encontrado na lista.");
}

console.log(`Total de etapas (tentativas): ${resultado.etapas}`); //Note
que o computador precisou de 7 etapas para isolar o item, e mais uma etapa
para confirmar que ele é o item procurado, totalizando 8 etapas.
```

IMPORTANTE Note que o computador precisou de **7 etapas** para isolar o item e mais **1 etapa** para confirmar que ele é o item procurado, totalizando 8 etapas. Utilizamos, na complexidade algorítmica, o número de etapas para chegar até o número, "desprezando" esta etapa final.

1.3 Encontrar número de telefone por nome em agenda (tempo de execução em Big O). Resposta: $O(\log n)$. Como uma agenda telefônica está ordenada alfabeticamente pelos nomes, você pode usar a pesquisa binária.

1.4 Encontrar o dono por um número de telefone na agenda. Resposta: $O(n)$. A agenda não está ordenada por números de telefone. Você não pode usar pesquisa binária, precisando fazer uma pesquisa linear (olhar item por item até achar).

1.5 Ler o número de cada pessoa da agenda. Resposta: $O(n)$. Você precisa acessar todos os elementos da lista uma vez.

1.6 Ler os números apenas dos nomes que começam com A. Resposta: $O(n)$. Na notação Big O, nós ignoramos as constantes e frações. Mesmo que você olhe apenas para, digamos, $1/26$ da lista (uma letra do alfabeto), o tempo de execução ainda cresce proporcionalmente ao tamanho total da lista.

Capítulo 2: Ordenação por seleção (Arrays vs Listas Encadeadas)

2.1 Aplicativo de finanças com muitas inserções e poucas leituras. Array ou lista encadeada?

Resposta: Lista encadeada. Inserir itens em uma lista encadeada é uma operação muito rápida ($O(1)$) se você inserir no início, pois não exige mover os outros elementos na memória, diferentemente do Array (onde adicionar itens quando o espaço acaba pode ser demorado). Como as leituras são poucas (leitura em lista encadeada é $O(n)$), a vantagem na inserção compensa.

2.2 Fila de pedidos de restaurante (inserir no final, remover do início). Array ou lista encadeada?

Resposta: Lista encadeada. Se você usar um array e remover um pedido do começo, todos os outros pedidos do array precisarão ser deslocados uma posição para a esquerda na memória, o que leva tempo $O(n)$. Em uma lista encadeada, remover do início (e inserir no fim) requer apenas a atualização dos ponteiros ($O(1)$), sendo muito mais eficiente.

Exemplo em Javascript (Fila com Array vs Fila com Lista Encadeada):

```
// O problema do Array para filas: remover do início (shift) é lento
let filaArray = ["Pedido1", "Pedido2", "Pedido3"];
// O método shift() reorganiza TODOS os índices seguintes (O(n))
let pedidoPronto = filaArray.shift();

// Uma Lista Encadeada Simples para Fila
class No {
  constructor(valor) {
    this.valor = valor;
    this.proximo = null;
  }
}

class FilaListaEncadeada {
```

```
    constructor() {
        this.inicio = null;
        this.fim = null;
    }

    // Inserir no final (O(1))
    enfileirar(valor) {
        const novoNo = new No(valor);
        if (!this.fim) {
            this.inicio = this.fim = novoNo;
            return;
        }
        this.fim.proximo = novoNo;
        this.fim = novoNo;
    }

    // Remover do início (O(1)) - Rápido, sem reorganizar memória!
    desenfileirar() {
        if (!this.inicio) return null;
        const valorRemovido = this.inicio.valor;
        this.inicio = this.inicio.proximo;
        if (!this.inicio) this.fim = null;
        return valorRemovido;
    }
}

const fila = new FilaListaEncadeada();
fila.enfileirar("Pedido 1");
fila.enfileirar("Pedido 2");
console.log(`Cozinhando: ${fila.desenfileirar()}`); // Saída: Cozinhando:
Pedido 1
```

Capítulo 3: Recursão e a Pilha de Chamadas (Call Stack)

3.1 Quais informações você pode retirar baseando-se apenas em uma pilha de chamada (call stack)?

Resposta: Você pode descobrir quais funções estão sendo executadas no momento, qual função chamou a função atual (a ordem de execução) e qual é o estado atual das variáveis locais de cada uma dessas funções paradas na memória, aguardando as outras terminarem.

3.2 O que acontece com a pilha quando a função recursiva fica executando infinitamente? Resposta:

A pilha cresce infinitamente até o computador ficar sem memória para alocar novos "blocos" (frames) de função. Quando isso acontece, o programa trava e emite um erro conhecido como **Estouro de Pilha** (*Stack Overflow*).

Exemplo em Node.js (Simulando um Stack Overflow):

```
// Uma função recursiva sem caso-base (condição de parada)
function recursaoInfinita(contador) {
    // Se ativarmos isso, ela vai rodar até a memória da pilha esgotar
    return recursaoInfinita(contador + 1);
}
```

```
}

try {
  console.log("Iniciando recursão infinita...");
  recursaoInfinita(1);
} catch (erro) {
  // O Node.js irá capturar o erro de esgotamento de memória da call
  stack
  console.error("Erro capturado:", erro.message);
  // Saída: Erro capturado: Maximum call stack size exceeded
}
```